PYTHON FOR VIDEO GAMES WITH PYGAME · LESSON 7

The final boss and your own mod

Goal: By the end you can move a character in a circle with sine and cosine, add a boss with its own health and a win condition, and read the original game's code to modify it.
Before you start:

- Lesson 6 - the game with menu, pause and game over.

To copy the code: when you copy from a PDF, many viewers drop the spaces at the start of each line, and in Python those spaces matter. Every step is also a `.py` file in the same folder as this guide: open it and copy from there.

Step by step

The final boss and your own mod

Last lesson: your game needs an ending, and every good ending has a boss. First you will make the boss move in circles, with a bit of math. Then you give it health, and a way to beat it. And finally you open the original game, Most Wanted UFO Classic, to recognize what you learned and modify it.

Step 2

Let's start at the top. On the first line, import math, Python's math module: that is where you will get sine and cosine from. Next to the other images, load the boss image, boss_image, with convert_alpha, so its transparent background stays transparent. After the Enemy class, create the Boss class, which is also a Sprite. scale2x makes its image twice as big, so the boss looks imposing, and get_rect gives it a rectangle to place it and to detect collisions. It also keeps two values of its own: an angle that starts at zero, and ten health points, because you do not beat a boss with a single shot.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-1.py`):

```
import math
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
boss_image = pygame.image.load("assets/boss.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
```

```
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


class Boss(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.transform.scale2x(boss_image)
        self.rect = self.image.get_rect()
        self.angle = 0
        self.health = 10


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
```

```
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 3

Now, inside the Boss class, add the update method. That is where the trick is: every frame, the angle grows by three hundredths. That angle says how far around its lap the boss is, and a full lap measures about six point two eight. Picture a point going around a circle. The cosine of the angle says how far right or left it is, from minus one to one, and the sine says how far up or down. You multiply the cosine by two hundred fifty and the sine by sixty, and add them to the point four hundred, one hundred fifty: the boss travels around a wide, flat oval at the top of the screen. At sixty frames per second, it completes one lap every three and a half seconds.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-2.py`):

```
import math
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
boss_image = pygame.image.load("assets/boss.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


class Boss(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.transform.scale2x(boss_image)
        self.rect = self.image.get_rect()
        self.angle = 0
        self.health = 10

    def update(self):
```

```
        self.angle += 0.03
        self.rect.center = (400 + math.cos(self.angle) * 250,
                            150 + math.sin(self.angle) * 60)


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
```

```
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 4

Now, the fight. Before the loop, next to lives, create boss with the value None, which means nothing in Python: when the game starts, there is no boss yet. In the Enter block that starts a game, set boss back to None. If you do not, the new game would start with the boss from the previous one. Now the logic, inside playing, right after the enemy collisions. If there is no boss yet and you reach one hundred points, you create it; checking for None keeps you from creating a new one every frame. If the boss exists, you move it with update, and with collidelist you look for a bullet touching it. If one does, that bullet disappears, the boss loses one health point, and the hit sound plays. When its health reaches zero, you get five hundred points, and state becomes won, so the game stops.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-3.py`):

```
import math
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
```

```
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
boss_image = pygame.image.load("assets/boss.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


class Boss(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.transform.scale2x(boss_image)
        self.rect = self.image.get_rect()
        self.angle = 0
        self.health = 10

    def update(self):
        self.angle += 0.03
        self.rect.center = (400 + math.cos(self.angle) * 250,
                            150 + math.sin(self.angle) * 60)


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
boss = None
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
```

```
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                boss = None
                for enemy in enemies:
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if boss is None and score >= 100:
            boss = Boss()
        if boss is not None:
            boss.update()
            hit = boss.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                boss.health -= 1
                hit_sound.play()
                if boss.health <= 0:
                    score += 500
                    state = "won"
        if lives <= 0:
            state = "game_over"
```

```
    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 5

Now the boss has to be seen. In the drawing, after the UFO, draw the boss only if it exists and you have not won yet. That way, when you beat it, it disappears from the screen. And at the end, just like the game over screen, if state is won, write YOU WIN with the big font, and the score below it. Enter works here too, because it starts a game from any state that is not playing or paused.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-4.py`):

```
import math
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
boss_image = pygame.image.load("assets/boss.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
```

```
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


class Boss(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.transform.scale2x(boss_image)
        self.rect = self.image.get_rect()
        self.angle = 0
        self.health = 10

    def update(self):
        self.angle += 0.03
        self.rect.center = (400 + math.cos(self.angle) * 250,
                            150 + math.sin(self.angle) * 60)


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
boss = None
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                boss = None
                for enemy in enemies:
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"

    if state == "playing":
```

```
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if boss is None and score >= 100:
            boss = Boss()
        if boss is not None:
            boss.update()
            hit = boss.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                boss.health -= 1
                hit_sound.play()
                if boss.health <= 0:
                    score += 500
                    state = "won"
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        if boss is not None and state != "won":
            screen.blit(boss.image, boss.rect)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
```

```
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    if state == "won":
        draw_centered("YOU WIN!", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 6

Run the game, and play a full game. When you reach one hundred points the boss appears, circling at the top. Each hit takes away health; the enemies keep falling, so stay alert. And when its health reaches zero: you win.

Try it:

```
python game.py
```

Step 7

Most Wanted UFO Mini is based on Most Wanted UFO Classic, an open source game. This is its main file: look at how the original does the boss. Look at these lines: it is the boss, moving in a circle. You already recognize the pieces: an angle that grows every frame, the screen center, a radius of one hundred, cosine for x and sine for y. The only new thing is math.radians: here the angle is in degrees, and that function converts it to radians, the same unit you used for your boss. A full lap is three hundred sixty degrees, or about six point two eight radians. The original is written differently, with everything inside one big function, but the ideas are the same ones you learned.

The original game's code is in `original/main.py`, in the course files.

You finished Most Wanted UFO Mini

You finished the course, and you built a complete game, starting from an empty window. You used the game loop, the keyboard, lists, and classes. You added collisions, score, lives, and states. And you moved a boss with sine and cosine. Your final project is a mod: download Most Wanted UFO Classic, and change something about the boss, like its radius, its speed, or its health. The art and sounds are free to use, as long as you credit their author. Thanks for playing, and for coding.

- Loop, keyboard, lists and classes
- Collisions, score, lives and states
- Sine and cosine to move in a circle
- Your mod: change the original game

Summary

- `math.cos(angle)` and `math.sin(angle)` go from -1 to 1; multiplied by a radius and added to a center, they trace a circle (or an oval, with different radii).

- If the angle grows a little every frame, the character circles; `0.03` per frame is one lap every 3.5 seconds at 60 FPS.
- `boss = None` means "there is no boss yet"; `if boss is not None:` avoids using it before it exists.
- The boss has its own health (`self.health`); when it reaches 0 the state becomes `"won"`.
- Most Wanted UFO Classic uses the same ideas in `main.py`, lines 166 to 170: `math.cos` and `math.sin` with a radius of 100.
- **For your mod:** the original game's code is MIT and its art and sounds are CC BY 4.0: you can modify and publish them as long as you credit Jose Daniel Soto.

Exercise

Final project: your mod.

- Download Most Wanted UFO Classic: `git clone https://github.com/Magic-Lab-Studio/most-wanted-ufo-classic`.
- Create an environment, install what `requirements.txt` asks for, and run `python main.py`.
- Open `main.py` and find the boss (lines 166 to 170). Change at least one thing: the radius (`100`), the spin speed (`self.boss_speed` in `game_manager.py`) or the boss health (`self.boss_health`).
- In your Most Wanted UFO Mini, make the boss shoot: every 60 frames add a bullet that falls from `boss.rect.midbottom`, and if it touches the UFO, take away a life. Hint: the original already does it in lines 172 to 175 of `main.py`.
- If you publish your mod, give credit: "Based on Most Wanted UFO Classic, by Jose Daniel Soto (CC BY 4.0)".

Solution

For step 4, in Most Wanted UFO Mini:

```
boss_bullets = []
boss_timer = 0
# inside if boss is not None:
boss_timer += 1
if boss_timer >= 60:
    boss_bullets.append(bullet_image.get_rect(midtop=boss.rect.midbottom))
    boss_timer = 0
for b in boss_bullets:
    b.y += 6
hit = player.collidelist(boss_bullets)
if hit != -1:
    boss_bullets.pop(hit)
    lives -= 1
    lose_sound.play()
boss_bullets = [b for b in boss_bullets if b.top < 600]
```

Put `boss_bullets = []` and `boss_timer = 0` before the loop, next to `boss = None`, and repeat them in the Enter block that starts a game. Draw each bullet in `boss_bullets` along with the others.

Based on Most Wanted UFO Classic, by Jose Daniel Soto: MIT code, CC BY 4.0 art and sounds.
A Magic Lab Studio course · magiclabstudio.com